

先把知识准备好，再让访客走进来。

AIHOT 小白指南 × GeniusQI 网站应用报告

12 个问题彩色图解动手验证新增项目分轨

最重要的不是复制一个资讯站，
而是把内容生产移出浏览的等待路径。



你会带走什么

内容层

仓库、学习笔记与版本证据整理成公开快照，让五种主题说同一件事。

性能层

已有 Three.js 不推倒；把下载、解析、GPU 准备分别量出来再优化。

体验层

作品先可读、链接先可点；三维探索可选，不用“看到加载条”冒充可操作。

可信度

个人实现与上游 Fork 分开，README 描述与真正跑通的 Demo 分开。

上游：KKKKhazix/AIHOT；固定源码 3343fe2b20db4be7269113752d82d3992fc52b6b。网站基线：my_website 本地 4ce04f8。方法：用户提供的 learn-project + 性能建议采用 huashu-flash。

范围说明

源码已克隆、核心阅读与离线实验已执行。未启动全套 PostgreSQL / Docker 服务，未使用模型 Key 或产生 API 费用；没有修改、部署 geniusqi.com。

01 这个网站是什么？它不是哪一类模板？

AIHOT 是“行业资料的编辑工厂”：采集信源，判断价值，形成事件与刊物，再展示给读者。不是三维渲染引擎，也不是给作品集换一套皮肤的模板。



摸底看到三个进程：web 展示页面，api 提供数据，worker 执行采集与模型任务。Node.js、React Router、Fastify、PostgreSQL 与任务队列各自承担工作。

对 GeniusQI：借鉴数据生产和展示边界，继续保留现有 Three.js 世界；不用为了学习它把 vinext/React 换成另一套框架。

02 为什么浏览时不用等 AI？

它把耗时的模型工作提前到后台。SSR（服务端渲染）是服务器先输出已有内容，不是等浏览器下载完整应用后再找内容。



源码约束是“页面不调模型”：worker 先保存结果，web 只经 HTTP 读 api。模型慢或临时不可用，不应把读者卡在生成动画里。

我们可把仓库介绍、学习摘要、版本日志提前审核并生成 JSON/Markdown。打开作品图鉴先有结果，再按用户选择进入三维。现有聊天是主动请求，不应绑在首页加载上。

03任务失败重试，会重复花钱吗？

任务队列是给后台工作排队的系统。重试很必要，但如果每次都重新调用付费模型，会把同一条摘要付费多次。



AIHOT 为付费请求记录回执，先存服务结果再供后续流程复用。预算熔断则为单位时间费用设置上限；它是成本护栏，不是无限免费。

网站的自动项目摘要、未来向导都可以学习这套规则。仍需处理“请求已发出但结果未知”的状态；不能把有回执简单说成永远只付一次。

04同一内容打两次分就可信吗？

评分是按规则对内容价值做判断，不是证明新闻事实。AIHOT 用相同标准独立评分两次，平均结果与信源分级门槛比较。



KnowHow 指你对行业的判断经验，写在 prompts 中。评测集是人工标注的“该选 / 不该选”样本；查准率、查全率帮助发现标准偏松还是偏严。

我们可评测“哪些项目放首页”，但保留人工确认。两次评分不能消除同模型的共同偏差；项目归属、可运行状态和原创贡献要靠仓库与演示证据。

05文章去重和事件归组有什么区别？

相同 URL 的重复采集是一种重复；不同媒体报道同一发布，则是不同文章、同一事件。只按链接判重会让首页充满一件事。



AIHOT 先用向量找候选，再判断是同一件事、后续进展还是不同事件。热度按独立参与方计数，并有时间衰减，避免一家重复报道无限加分。

作品集可以类比“项目→版本→证据”：README、提交、截图、演示同归一个项目。这里是移植建议，不是把新闻聚簇算法直接当项目管理器；也不按提交次数假造作品热度。

06如何避免网站、RSS、API 各说各话？

公开读取层就是所有公开出口共用的内容闸门。发布、隐藏、撤回、许可等规则只定义一次，而不是每个页面自己过滤。



AIHOT 的 `publication/scope.ts` 定义边界，网页、RSS、API、MCP 等从同一个 `publication` 层读。旧热点也在输出前检查是否仍可公开。

GeniusQI 的作品图鉴、五个主题、向导上下文也应共享同一份公开项目快照。私有仓库不是“加隐藏 CSS”就安全，必须在构建输出前剔除。

07 个人化内容和引擎要放在一起吗？

AIHOT 把站名、信源、分类、门槛、提示词集中在 industry 配置包里；通用业务流程与行业判断分开。



配置驱动意味着调整内容不必重写渲染器。但配置仍有结构约束和验证，并不是任意一段自然语言都能自动变成可靠功能。

我们可分成 content/projects、content/studies、world-layout 与 rendering。新增作品先增加项目记录，再给某个主题分配展台；别每次手改五套世界代码。

08 缓存越久，网站就越好吗？

缓存是复用已有响应，减少重复工作；但缓存太久会展示已撤回内容，或延迟刚达到发布时间的内容。速度和正确性必须一起考虑。



AIHOT 的 releaseBoundCache 取多个截止时刻中的最早一个。普通上限 60 秒、5 秒后有新内容，则只缓存 5 秒；上游 no-store 时不能扩大为公共缓存。

本次直接隔离并执行了该上游函数，6 个用例通过。我们可让带内容哈希的模型长期缓存，目录快照短期缓存；不要原样照搬新闻站的所有 TTL。

09可以把所有新仓库都写成“我的作品”吗？

公开性、仓库归属和个人贡献是三个不同问题。你拥有一个 Fork，并不意味着上游软件的全部能力都是你创造的。



本次读取 GitHub 元数据和街区猫王 README。6 个候选是 Fork，1 个是非 Fork；这些是仓库证据，还不是本人贡献审计或实际 Demo 成功证据。

建议区分“个人实现 / 上游学习 / 有证据的二次改造 / 概念”。街区猫王可优先做试玩，标注桌游改编与规则来源；私有仓库没有授权过的内容不输出。

10可以直接学它做自动更新吗？

后台定时采集可以更新资讯，但自动化也会增加 API 成本、内容误判、版权和运维负担。自动化不是本报告擅自开启的功能。



更适合你的第一步是“审核后生成静态内容”：整理项目、来源、解决问题、截图与状态，再输出同一份公开清单。现有同步脚本有基础，但还缺学习/改造轨道。

以后你明确需要持续同步时，再接 Release、工作日志或知识库摘要；保留审批记录、预算上限和失败告警。只展示有授权的摘要与原文链接，非授权全文不搬运。

11 它的页面设计有什么可以学？

从源码看，主页先呈现热点与内容，再提供分类和搜索；手机与桌面有不同布局。它的价值是信息层级，而不是堆满大标题与特效。



GeniusQI 可以先回答“你做了什么、解决谁的问题、哪里能体验”，再提供探索世界。3D 是可选的深度浏览，不应成为理解作品的唯一通道。

具体建议：作品卡先显示问题和证据；学习卡显示主实验与指南；版本卡显示最近验证时间。官网在线抓取本次超时，以上视觉判断来自开源页面代码，非实机全站体验。

12 能把它的“毫秒级”速度搬到我们场景吗？

不能。接口或服务端耗时不等于浏览器能操控人物的耗时。三维还需要传输模型、解压、解析、贴图上传、着色器编译和碰撞准备。



我们已有 Three.js ^0.185.1、GLTFLoader、VRM 插件、按主题懒加载、移动人物和 gzip 下载。上轮晴岚浮岛配对实验 p75 为 6.62→5.83 秒，日期是 2026-09-27。

那是旧版实验室对比，不能当今天的手机速度。本次只做源码与资源审计，没有新性能 A/B；不能宣称再次提速，也没有理由先重写为 R3F 或 WebGPU。

它给我哪些能力？

学方法，不把资讯站整个搬进作品集。

源码经验

→

个人知识库

→

GeniusQI 内容系统

01 • 项目内容生成器

仓库资料 → 人工核对问题、贡献、状态 → 公开 JSON/Markdown。一次准备，首页、图鉴、世界和向导共同读取。

02 • 学习成果展厅

每项学习挂指南、主实验、输出证据与上游链接，形成“我如何学会并应用”的个人记录，而非上游项目转售橱窗。

03 • 开发版本时间线

Release、提交和截图归到项目版本。分清“构思”“代码已提交”“已验证”“已上线”，让访客知道哪些能真正体验。

04 • 来源和公开范围的统一闸门

公开范围与许可在生成阶段确定；私有源、未确认文案不进入公共 bundle，撤回同时影响所有主题与导出。

05 • 可控的 AI 内容后台

将来需要自动摘要时，用队列、请求回执、预算和评测。是否自动更新、用哪家模型、预算多少，留给你明确决定。

移植边界

目前不需要 PostgreSQL + pg-boss 全家桶，不新建行业新闻站，不照搬 AIHOT 评分门槛，不复制其名字与 Logo。

新增项目怎么展示才像你？

截至 2026-10-03 的公开仓库核对；列表只包含本次研究的 7 个候选，不是全账号统计。

项目	建议身份	上游 / 证据
AIHOT	学习与改造	KKKKhazix/AIHOT
street-cat-king	个人实现候选	非 Fork；桌游改编实现
floorplan-3d	学习与改造	wy51ai/floorplan-3d
crush-monitor	学习与改造	FerryCorleone/crush-monitor
laya	学习与改造	NandhaKishorM/laya
magpie	学习与改造	yetone/magpie
BeefTV	学习与改造	glanderness/BeefTV

优先做：街区猫王试玩展台

README 描述了 2-4 人本地轮流与 AI 对手、浏览器存档、零依赖前端结构。适合 “空间里的棋盘 → 点击 → 隔离试玩 → 查看规则取舍 → 仓库” 。

尚未实机验证游戏。 不能把 README 里的部署方法说成线上 Demo 已可用。桌游规则来源与改编身份也要标清。

Fork 要如何升级为个人作品？

补一条能定位的改动证据：你的 commit / diff、改造前后截图、运行测试与明确使用场景。没有这些时显示 “上游学习” ；有证据再显示 “二次改造” 。非 Fork 也不自动证明所有素材或玩法原创。

未做的事

私有仓库没有自动公开。没有检查所有 Fork 的个人分支，因此没有断言 “你没有改过它们” 。本次只生成候选清单，没有扩建正式场景。

Three.js：要优化，不必重写。

确认：网站已使用 Three.js ^0.185.1 + VRM。按主题懒加载、移动版模型、gzip、字节缓存和下载并行已有；不能把这些再次算作新优化。

当前人物资源	gzip 字节量 ¹	内嵌图片数	4Mbps 理想传输 ²
lite.vrm	1.34 MB	19	2.68s
male-lite.vrm	1.47 MB	25	2.95s
male-web.vrm	1.99 MB	25	3.99s
web.vrm	1.54 MB	19	3.08s

¹ MB=1,000,000 字节；实际传输还取决于响应头与路径。² 公式=文件字节×8÷4,000,000，只是估算，不含延迟、竞争、解压、VRM 解析与 GPU 工作，不是手机实测。

按优先级做下一轮实验

P0 • 加分段时间戳：主题点击 → JS 到达 → 人物下载 → 解压 → VRM 解析 → 环境/碰撞 → 首帧 → 输入响应。记录真正可操作的终点。

P1 • 贴图/几何资产实验：先量 CPU/GPU 和字节贡献，试 KTX2/Basis 或 Meshopt。人物含表情、骨骼与 VRM 扩展，必须验证材质、动作、头发和解码兼容；不能只看下载减小。

P1 • 首帧准备：目前场景模块没有 compileAsync 调用。可用异步编译、贴图预准备做对照；这会移动开销，不保证端到端更快，也不能假定所有设备支持相同扩展。

P2 • 运行流畅度：InstancedMesh、共享材质、视锥裁剪、静态阴影、像素预算可能改善 FPS；优先保持现有交互，别把帧率改善说成进入时间改善。

验收：前后各至少 10 次同条件交替 A/B，报 p50/p75/p95；检查五主题、男女角色、二段跳、桌面与手机布局。CDN/DNS 改动单独授权。默认减半是实验目标，不是承诺。

主实验 / 先产出内容，再读快照

纯 Node 实验，使用已克隆的真实上游函数 + 自建公开快照材料。无需 API Key、数据库或付费服务。文件放在 exercise/，不改 repo/。

1 • 固定环境和工作目录

```
$node = "C:\Users\Windows\cache\codex-runtimes\codex-primary-runtime\dependencies\node\bin\node.exe"
Set-Location -LiteralPath 'F:\reminder\aihot'
& $node --version
git -C .\repo log -1 --format="%H"
```

目的：确认同一份代码。验证：Node v24.19.0, 3343fe2...；本次 Python 使用进程级 UTF-8，没有改系统编码。

2 • 验证架构边界

```
& $node .\exercise\architecture-windows.mjs
```

目的：验证 web 不碰数据库、公开出口只读公开层。验证：5/5 通过。原测试直接在 Windows 为 3/5：两项用 POSIX 分隔符比较路径；实验夹具只归一化反斜杠，不改上游源码。

3 • 跑公开快照与缓存实验

```
& $node .\exercise\lab.mjs
```

目的：内容预处理与范围过滤。验证：11 个断言通过，输出 output/public-projects.json 和 lab-results.json；合成私有与草稿项被剔除。

4 • 查看可见中间成果

```
Get-Content -LiteralPath .\exercise\output\public-projects.json
```

验证：7 个候选，6 个学习/改造，1 个个人实现候选；每个项目都保留来源和 “Demo 未验证” 状态。

主实验 / 改参数，观察结果

5 • 在独立预览中比较两条轨道

```
Start-Process .\learning_lab.html
```

这是 HTML 学习实验，不是正式网站。点击“全部 / 个人实现 / 学习改造”观察项目数量；下方选择缓存场景，查看 60s、5s 与 no-cache 的差别。PDF 为静态，互动请打开旁边 HTML。

6 • 审计现有资产而非猜慢在哪

```
& $node .\exercise\audit-website.mjs
Get-Content -LiteralPath .\exercise\output\website-audit.json
```

验证：4 个 web/lite 模型；每个 gzip 解压后逐字节等于原 VRM。输出材质图片数与字节，不声称网站因此提速。

缓存实验的实际结果

场景	公共缓存结果
普通请求，上限 60 秒	s-maxage=60
5 秒后有新发布	s-maxage=5
上游只准缓存到 2 秒后	s-maxage=2
上游 no-store / 0 / 已过期	no-cache



测试不能替代的事情

11 个断言与 5 个架构测试不等于全站测试通过。没有运行新闻采集、模型评分、数据库迁移和完整后台；没有启动街区猫王试玩。学习实验显示概念可行，不等于已接入生产。

打开预览这一操作只启动本地学习 HTML；其余命令已实际执行并核对输出。自建脚本与 JSON 全部随学习成果保存，可复跑。

下一步：一条内容线，一条性能线。

内容线

先接街区猫王与学习成果。把公开范围、原创/改造身份、解决问题、验证日期、上游、Demo 状态做成字段；五主题共用。

性能线

先记录 click-to-controllable 分段，再做一项小实验。若主要卡传输就优化资产；若卡解析/首帧再动主线程/GPU。

不要同时重写框架、换渲染器、换托管。那样即使变快，也不知道是谁起作用。保留已验证二段跳和世界玩法；按同口径回归。

来源 / 可复核证据

固定源码与 README

<https://github.com/KKKKhazix/AIHOT/tree/3343fe2b20db4be7269113752d82d3992fc52b6b>

架构：三个进程、公开读取层、成本与安全阀

<https://github.com/KKKKhazix/AIHOT/blob/3343fe2b20db4be7269113752d82d3992fc52b6b/docs/architecture.md>

精选与校准：门槛、两次评分、人工金标样本

<https://github.com/KKKKhazix/AIHOT/blob/3343fe2b20db4be7269113752d82d3992fc52b6b/docs/selection.md>

真实缓存函数：releaseBoundCache

<https://github.com/KKKKhazix/AIHOT/blob/3343fe2b20db4be7269113752d82d3992fc52b6b/apps/web/app/lib/api.server.ts>

MIT 与品牌、字体等许可说明

<https://github.com/KKKKhazix/AIHOT/blob/main/NOTICE>

Three.js WebGLRenderer / compileAsync

<https://threejs.org/docs/pages/WebGLRenderer.html>

Three.js KTX2Loader

<https://threejs.org/docs/pages/KTX2Loader.html>

Three.js GLTFLoader / 扩展解码器

<https://threejs.org/docs/pages/GLTFLoader.html>

街区猫王：仓库与 README

<https://github.com/Smashwinny/street-cat-king>

本地证据

exercise/output/ 下保存实验结果、公开候选快照和资产审计；exercise/repository-evidence.json 保存公开仓库元数据。旧速度数据来自 my_website/perf/报告.md，明确标注 2026-09-27 的实验室条件。

未动：线上站点、DNS/CDN、模型 Key、私有仓库公开范围、上游源码与其他任务的笔记。研究建议未当作性能收益。